

Local GitHub Webhooks + ngrok + Auto Pull/Install/Run (Node.js)

Goal: expose a local webhook endpoint with **ngrok**, configure a **GitHub webhook** to call it, verify the webhook **secret signature**, then automatically **git pull**, **npm install**, and start a **React app** locally.

Prerequisites

- Node.js
 - Git
 - A GitHub repository containing a React app
 - ngrok account
-

1) Install and configure ngrok

1.1 Install ngrok from <https://ngrok.com/download>

After install, confirm:

```
ngrok version
```

1.2 Add your ngrok authtoken

In the ngrok dashboard, copy your authtoken, then run:

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

2) Create the webhook listener (Node.js)

We'll build a small Node server that:

1. listens on **localhost:9000/webhook/github**
2. verifies GitHub's **X-Hub-Signature-256** HMAC signature
3. runs a deploy script if it's a push to a target branch

2.1 Create project folder

```
mkdir webhook-listener  
cd webhook-listener  
npm init -y
```

```
npm pkg set type="module"
npm install express
```

2.2 Create `.env`

Create a file named `.env`.

We'll use these variables:

- `GITHUB_WEBHOOK_SECRET` (must match the webhook secret set in GitHub)
- `REPO_URL` (your repo URL)
- `DEPLOY_DIR` (where to clone/pull)
- `TARGET_BRANCH` (e.g. `main`)
- `APP_PORT` (port where the React app will run, e.g. `3000` or `5173`)

Example:

```
GITHUB_WEBHOOK_SECRET=replace_me
REPO_URL=https://github.com/your-org/your-react-repo.git
DEPLOY_DIR=../deployed-react-app
TARGET_BRANCH=main
APP_PORT=3000
```

In order to load `.env`, install `dotenv` and add `import "dotenv/config";` at the top of `server.js`.

2.3 Create `server.js`

```
import express from "express";
import crypto from "crypto";
import { spawn } from "child_process";
import "dotenv/config";

const app = express();

// IMPORTANT: to verify GitHub signatures you must use the *raw* request
// body bytes.
app.post("/webhook/github", express.raw({ type: "*" }), async (req, res) => {
  try {
    const secret = process.env.GITHUB_WEBHOOK_SECRET;
    if (!secret) return res.status(500).send("Missing GITHUB_WEBHOOK_SECRET");

    // GitHub sends the signature in X-Hub-Signature-256 when a secret is
    // configured. :contentReference[oaicite:3]{index=3}
    const sigHeader = req.header("X-Hub-Signature-256");
    if (!sigHeader) return res.status(401).send("Missing signature header");
```

```
const expected =
  "sha256=" +
  crypto.createHmac("sha256", secret).update(req.body).digest("hex");
const ok = crypto.timingSafeEqual(
  Buffer.from(sigHeader),
  Buffer.from(expected),
);
if (!ok) return res.status(401).send("Invalid signature");

// Parse JSON payload after signature verification
const payload = JSON.parse(req.body.toString("utf8"));

// Only act on push events
const event = req.header("X-GitHub-Event");
if (event !== "push") return res.status(200).send("Ignored event");

const targetBranch = process.env.TARGET_BRANCH || "main";
const ref = payload?.ref; // e.g. "refs/heads/main"
if (ref !== `refs/heads/${targetBranch}`) {
  return res.status(200).send(`Ignored branch ${ref}`);
}

// Trigger deploy script
const script = process.platform === "win32" ? "deploy.ps1" :
"deploy.sh";
const cmd = process.platform === "win32" ? "powershell.exe" : "bash";
const args =
  process.platform === "win32"
    ? ["-ExecutionPolicy", "Bypass", "-File", script]
    : [script];

const child = spawn(cmd, args, {
  env: process.env,
  stdio: "inherit",
});

child.on("exit", (code) => {
  if (code === 0) console.log("Deploy completed successfully");
  else console.error("Deploy failed with code", code);
});

res.status(200).send("Deploy triggered");
} catch (e) {
  console.error(e);
  res.status(500).send("Server error");
}
});

const port = Number(process.env.WEBHOOK_PORT || 9000);
app.listen(port, () => {
  console.log(`Webhook listener on
http://localhost:${port}/webhook/github`);
});
```

3) Create the deploy scripts

This does:

- clone if missing, otherwise pull
- install dependencies
- start the React app

Below is a generic approach that tries to run our react app, else prints a message.

3.1 macOS/Linux: `deploy.sh`

```
#!/usr/bin/env bash
set -euo pipefail

: "${REPO_URL:?Missing REPO_URL}"
: "${DEPLOY_DIR:?Missing DEPLOY_DIR}"
: "${TARGET_BRANCH:=main}"

echo "== Deploy started =="
echo "Repo: $REPO_URL"
echo "Dir: $DEPLOY_DIR"
echo "Branch: $TARGET_BRANCH"

if [ ! -d "$DEPLOY_DIR/.git" ]; then
    echo "Cloning..."
    git clone --branch "$TARGET_BRANCH" "$REPO_URL" "$DEPLOY_DIR"
else
    echo "Pulling latest..."
    git -C "$DEPLOY_DIR" fetch --all
    git -C "$DEPLOY_DIR" checkout "$TARGET_BRANCH"
    git -C "$DEPLOY_DIR" pull --ff-only
fi

echo "Installing dependencies..."
cd "$DEPLOY_DIR"
npm install

echo "Stopping previous app (best-effort)..."
# If you use a process manager (recommended), stop it here instead.
# This is intentionally minimal.

echo "Starting app..."
if npm run | grep -qE ' start'; then
    # if we built our React app with create-react-app
    nohup npm run start > app.log 2>&1 &
    echo "Started with: npm run start"
elif npm run | grep -qE ' dev'; then
    # if we built our app with vite
    nohup npm run dev -- --host 0.0.0.0 > app.log 2>&1 &
```

```
    echo "Started with: npm run dev"
else
    echo "No start/dev script found in package.json"
    exit 1
fi

echo "== Deploy done =="
```

Make it executable:

```
chmod +x deploy.sh
```

3.2 Windows (PowerShell): `deploy.ps1`

```
$ErrorActionPreference = "Stop"

if (-not $env:REPO_URL) { throw "Missing REPO_URL" }
if (-not $env:DEPLOY_DIR) { throw "Missing DEPLOY_DIR" }
if (-not $env:TARGET_BRANCH) { $env:TARGET_BRANCH = "main" }

Write-Host "== Deploy started =="
Write-Host "Repo: $env:REPO_URL"
Write-Host "Dir: $env:DEPLOY_DIR"
Write-Host "Branch: $env:TARGET_BRANCH"

$deployDir = $env:DEPLOY_DIR

if (-not (Test-Path "$deployDir\.git")) {
    Write-Host "Cloning..."
    git clone --branch $env:TARGET_BRANCH $env:REPO_URL $deployDir
} else {
    Write-Host "Pulling latest..."
    git -C $deployDir fetch --all
    git -C $deployDir checkout $env:TARGET_BRANCH
    git -C $deployDir pull --ff-only
}

Write-Host "Installing dependencies..."
Set-Location $deployDir
npm install

Write-Host "Starting app..."
# Start in background and write logs
$npmScripts = (npm run) | Out-String

# if we built our React app with create-react-app
if ($npmScripts -match "\sstart\s") {
    Start-Process -FilePath "npm" -ArgumentList "run", "start" -
    RedirectStandardOutput "app.log" -RedirectStandardError "app.err.log"
```

```
Write-Host "Started with: npm run start"
}
# if we built our React app with vite
elseif ($npmScripts -match "\sdev\s") {
    Start-Process -FilePath "npm" -ArgumentList "run","dev","--","--"
    host,"0.0.0.0" -RedirectStandardOutput "app.log" -RedirectStandardError
    "app.err.log"
    Write-Host "Started with: npm run dev"
}
else {
    throw "No start/dev script found in package.json"
}

Write-Host "== Deploy done =="
```

4) Run the webhook listener locally

Set env vars in your terminal, then run the listener.

macOS/Linux

```
export GITHUB_WEBHOOK_SECRET="your_secret_here"
export REPO_URL="https://github.com/your-org/your-react-repo.git"
export DEPLOY_DIR="$HOME/deployed-react-app"
export TARGET_BRANCH="main"
export WEBHOOK_PORT="9000"

node server.js
```

Windows (PowerShell)

```
$env:GITHUB_WEBHOOK_SECRET="your_secret_here"
$env:REPO_URL="https://github.com/your-org/your-react-repo.git"
$env:DEPLOY_DIR="$HOME\deployed-react-app"
$env:TARGET_BRANCH="main"
$env:WEBHOOK_PORT="9000"

node server.js
```

5) Expose the webhook endpoint with ngrok

In a separate terminal:

```
ngrok http 9000
```

ngrok will print a public HTTPS URL like:

```
https://abcd-1234.ngrok.app
```

Your webhook URL will be:

```
https://abcd-1234.ngrok.app/webhook/github
```

(ngrok supports managed domains / stable dev domains depending on your plan/settings. ([ngrok.com][2]))

6) Configure the GitHub webhook

In your GitHub repo:

1. Go to **Settings** → **Webhooks** → **Add webhook**
2. **Payload URL**: `https://YOUR_NGROK_DOMAIN/webhook/github`
3. **Content type**: `application/json`
4. **Secret**: set a strong secret (same as `GITHUB_WEBHOOK_SECRET`)
5. **Which events would you like to trigger this webhook?**
 - Choose **Just the push event** (recommended for this exercise)
6. Click **Add webhook**

GitHub will sign deliveries using your secret and include `X-Hub-Signature-256`. ([GitHub Docs][3])

7) Test end-to-end

1. Make a small commit and push to `main`
 2. GitHub sends the webhook to your ngrok URL
 3. Your local listener:
 - verifies signature
 - confirms event and branch
 - runs deploy script
 4. The React app starts locally (check logs)
-

8) Troubleshooting

Signature verification fails

- Ensure you are verifying **raw bytes** of the request body (not parsed JSON).
- Ensure GitHub webhook secret exactly matches `GITHUB_WEBHOOK_SECRET`.
- Use `X-Hub-Signature-256` (recommended). ([GitHub Docs][4])

ngrok URL keeps changing

- Use an ngrok dev domain / reserved domain if available in your account.
- Otherwise update the GitHub webhook URL after restarting ngrok.